```
object Manager {

  sealed trait Command                                                    ❶
  case class Delegate(texts: List[String]) extends Command                ❶
  private case class Report(description: String) extends Command          ❶

  def apply(): Behavior[Command] =
    Behaviors.setup { context =>
      val timeout: Timeout = Timeout(3, SECONDS)                          ❷

      Behaviors.receiveMessage { message =>
        message match {
          case Delegate(texts) =>                                         ❸
            files.map { file =>
              val worker: ActorRef[Worker.Command] =                      ❹
                context.spawn(Worker(file), s"worker-$file")
              context.ask(worker, Worker.Parse)(timeout) {                ❺
                case Success(Worker.Done) =>                              ❻
                  Report(
                    s"$text read by ${worker.path.name}")                 ❻
                case Failure(ex) =>
                  Report(
                    s"parsing '$file' has failed with [${ex.getMessage()}")
              }
            }
            Behaviors.same
          case Report(description) =>
            context.log.info(description)                                 ❼
            Behaviors.same
        }}}
}
```

❶ Protocol

❷ Timeout implicitly passed to ask

❸ List of files to parse

❹ Creates a worker

❺ Asks the worker to start parsing

❻ Handles the response by creating a report

❼ Prints the report

Now that you've seen how the `ask` works, you know the manager's entire workflow. The manager receives the text to parse, creates a